Design of a Web-Based Educational Platform for Collaborative Programming with CRDT and Hint Generation

Viktoriia Pugach
Higher School of Economics, Faculty of Computer Science
Moscow, Russia
ORCID: 0000-0002-9117-4879
vppugach@edu.hse.ru

***Abstract*—Educational organizations running online programming courses for children often rely on several disconnected tools, including child-oriented platforms, professional collaborative editors, and video conferencing systems. This fragmentation creates multiple points of failure, weak pedagogical feedback, and prevents the use of a single environment that offers both real-time collaborative code editing and in-context, child-adapted hint generation. Furthermore, the integration of CRDT-based synchronization with an AST-driven pedagogical hint engine under strict latency and reliability constraints has not been systematically addressed in the literature. This paper presents the design and planned evaluation of an educational web platform that unifies (1) conflict-free real-time code synchronization using CRDTs, (2) an AST-based hint service that detects novice errors and generates child-adapted explanations, and (3) session and learning management within a single system. The architecture is structured using Domain-Driven Design with three bounded contexts and implemented as a modular monolith with a dedicated real-time synchronization service for horizontal scaling. The main scientific contribution is a documented architectural approach for integrating the CRDT collaboration layer with a pedagogical hint engine—a combination not found in existing systems. The paper describes the methodology, system design, technology choices, and the planned evaluation strategy. Evaluation will assess synchronization latency, hint response time, CRDT convergence, and usability for the target audience. The approach is intended for sovereign or on-premises deployment and may support further research in EdTech and CSCL.**

Keywords—Collaborative programming, CRDT, real-time synchronization, educational technology, intelligent tutoring systems, hint generation, software architecture, Domain-Driven Design

I. Introduction

A. Relevance for Software Engineering and EdTech

Interactive learning platforms that support hands-on, collaborative programming are increasingly important. Effective learning relies on live collaboration, peer programming, and project-based activities [1], [2]. Many institutions, however, rely on a patchwork of tools: child-oriented platforms (e.g., Scratch, Code.org), professional collaborative IDEs (e.g., VS Code Live Share, JetBrains Code With Me), and communication tools (e.g., Zoom, Discord). Child-focused platforms rarely provide true real-time collaborative code editing; professional tools are not designed for young learners and lack age-appropriate feedback and gamification. Using several unintegrated applications increases technological risk, cognitive load, and makes it difficult to provide timely, context-aware help. Dependence on external or offshore platforms also raises sustainability and sovereignty concerns for educational organizations [3]. A single environment that combines reliable real-time collaboration, built-in pedagogical support, and the possibility of sovereign or on-premises deployment is therefore needed.

B. Problem statement

The central problem is the absence of a unified, methodologically and technically coherent digital environment for online programming lessons for children. Current practice forces educators to assemble four to five heterogeneous tools. This leads to (1) high sensitivity to failures and provider lock-in, (2) weak pedagogical feedback and fragmented interaction, and (3) a lack of studied solutions that integrate seamless state synchronization with semantic analysis and adaptive hint generation under the latency constraints of a live lesson. The technical and research challenge is to design an architecture that unifies collaborative editing, hint generation, and learning management in one system and to document this integration pattern for the community.

C. Brief Overview of Existing Approaches and Their Limitations

Child-oriented platforms (Scratch, Code.org, Tynker) offer structured courses and gamification but typically lack real-time collaborative code editing. Professional collaborative tools provide strong real-time editing and sometimes awareness (cursors, selections) but are not adapted for children and do not offer pedagogical hint generation. Intelligent tutoring systems for programming address feedback and hints [6], [7] but are not integrated with CRDT-based collaborative editors. Thus, the main directions—collaborative editing (including CRDTs),

ITS/hint generation, and child-oriented EdTech—remain separate; no existing system combines them in one platform with strict latency and reliability requirements.

D. Goal and Objectives of the Paper

The aim is to design and develop an architecture and prototype of a web platform for collaborative programming education that integrates CRDT-based synchronization with a system for generating adaptive, child-oriented hints. The objectives are: (1) to design and justify an architecture that ensures conflict-free convergence and low sync latency; (2) to embed context-dependent hint generation (AST-based error detection and child-adapted language) into the collaboration loop; (3) to provide a clear, modular architecture suitable for on-premises or sovereign deployment; and (4) to define and carry out an evaluation plan (experiments and testing) to validate the design. Within the article, we focus on documenting the problem, the research gap, the proposed solution, and the planned evaluation.

E. Scientific and practical significance, and contributions

This work contributes to the intersection of software engineering and computer-supported collaborative learning by proposing an architectural approach for integrating real-time collaborative programming with pedagogically meaningful feedback.

From a scientific perspective, the main contribution is the formalization of an integration pattern that connects a CRDT-based collaboration layer with a semantic code-analysis pipeline responsible for generating pedagogical hints. While CRDTs have been extensively studied in the context of distributed collaborative editors, and intelligent tutoring systems have addressed automated feedback generation, the two research directions have largely evolved independently. The proposed architecture demonstrates how these components can be combined within a single system while preserving strict performance requirements necessary for synchronous online learning.

A second contribution concerns the explicit treatment of pedagogical constraints as architectural requirements. Instead of adding tutoring functionality as a post-processing layer, the design integrates hint generation directly into the collaborative programming workflow. This allows error detection, contextual hint generation, and collaborative editing to operate within the same interaction loop.

Third, the system introduces a domain-driven decomposition of the platform into three bounded contexts—Collaborative Editing, Pedagogical Analysis, and Learning Management. This separation enables independent evolution of collaboration infrastructure and pedagogical services while maintaining clear architectural boundaries.

Finally, the work proposes an evaluation framework that jointly assesses system performance and pedagogical usability. The framework combines distributed systems metrics (synchronization latency, CRDT convergence) with educational indicators such as usability (SUS), clarity of hints, and user engagement.

The project is carried out in collaboration with the educational organization “Inzhinium,” which provides domain expertise and serves as a partner for pilot evaluation of the platform in real teaching scenarios.

F. Structure of the Paper

Section II reviews related work and states the research gap. Section III presents the proposed approach, models, system design, and architecture. Section IV describes implementation, tools, and technologies. Section V sets out the planned experiments, testing strategy, metrics, and how results will be presented and analyzed. Section VI discusses limitations, threats to validity, and applicability. Section VII concludes and outlines future work.

II. Related Work

A. Collaborative editing and CRDTs

Conflict-Free Replicated Data Types (CRDTs) guarantee eventual consistency without central coordination and support offline operation and arbitrary order of message delivery [1]. They have been applied to collaborative text editing in systems such as Automerge and Yjs [2]. In several Operational Transformation (OT) designs, a central server is required to perform transformation of operations and to broadcast them, which introduces a single point of failure and extra latency under poor network conditions [3]. CRDTs do not require the server to perform transformation; a causally-ordered communication layer is sufficient, and merge is performed on the client. This property makes CRDTs a better fit for unstable networks and for sovereign or on-premises deployment, where minimizing dependence on a central transformation server is important. Our design uses Yjs (YATA-based CRDT) for the shared code document, with a WebSocket layer and Redis Pub/Sub for multi-node scaling. The literature does not, however, address the joint design of such a CRDT-based real-time layer with an in-loop pedagogical hint engine for children.

B. Intelligent tutoring and hint generation

Intelligent Tutoring Systems (ITS) for programming provide adaptive feedback and hints [4], [5]. Static analysis of novice code is used to detect typical errors and to drive feedback [6]. Approaches range from rule-based and template-based hint generation to LLM-based explanations. For a live, low-latency classroom setting, we combine local AST-based analysis (e.g., Tree-sitter) with template- and optionally LLM-based hint generation. The hint text is adapted for children (short, concrete, non-technical language, friendly tone), and hints are visually tied to the error location. Existing ITS and hint-generation work is not integrated with CRDT-based collaborative editors in a single platform.

C. Educational platforms and CSCL

Child-oriented platforms (Scratch, Code.org, Tynker) offer structured courses and gamification but usually lack real-time collaborative code editing [1], [2]. Professional collaborative tools provide strong collaboration but are not designed for children and do not offer pedagogical hint generation. Computer-Supported Collaborative Learning (CSCL) emphasizes shared tasks and peer interaction [7]; our platform is designed to support this by combining a single shared code document (CRDT), awareness (cursors and selections), and in-context hints. Domain-Driven Design (DDD) is used to separate

collaboration, pedagogy, and learning-management concerns [8], [9].

D. Classification and critical analysis

Existing solutions can be classified into: (A) child-oriented platforms (strong pedagogy, weak or no real-time collaboration); (B) professional collaborative editors (strong collaboration, no child-oriented pedagogy or hints); (C) ITS and hint systems (strong feedback logic, not integrated with real-time collaborative editing). None of these classes provides an integrated environment that combines CRDT-based real-time sync, AST-driven hint generation, and child-centered design in one system with defined latency and reliability targets.

E. Research Gap and Research Questions

Existing solutions for programming education can be broadly divided into three categories.

The first category includes child-oriented educational platforms such as Scratch, Code.org, and Tynker. These systems provide structured learning environments, visual programming tools, and gamification mechanisms aimed at novice learners. However, they typically lack robust real-time collaborative editing capabilities required for synchronous programming activities.

The second category consists of professional collaborative development tools, including JetBrains Code With Me, VS Code Live Share, and Replit. These tools provide advanced collaboration features such as shared editors, presence awareness, and sometimes integrated communication tools. Nevertheless, they are primarily designed for professional developers and do not provide pedagogically adapted feedback mechanisms or child-oriented user interfaces.

The third category includes intelligent tutoring systems and automated feedback tools that analyze student code and generate hints or suggestions. Such systems provide pedagogical support through static analysis or rule-based hint generation. However, they are typically implemented as standalone tutoring environments and are rarely integrated with real-time collaborative editing infrastructure.

To illustrate these differences, Table I compares representative platforms across key capabilities relevant to collaborative programming education, including real-time editing, pedagogical hints, and support for child-adapted interfaces.

TABLE I. COMPARISON OF EXISTING PLATFORMS AND THE PROPOSED SOLUTION

Criterion	***Code With Me***	***Code.org***	***Scratch***	***Replit***	***Tynker/ Blockly***	***Proposed Solution***
Real-time collaborative editing	✓	—	partial	✓	—	✓
Child-adapted interface	—	✓	✓	—	✓	✓
Adaptive pedagogical hints	—	✓	✓	—	✓	✓
Integrated A/V communication	✓	—	—	—	—	✓
Gamification	—	✓	—	partial	✓	✓
AI-assisted hint generation	✓*	—	—	✓	—	✓
On-premises / sovereign deployment	—	✓	—	—	—	✓

The comparison shows that existing solutions usually address only one or two of these dimensions. Educational platforms focus on pedagogy but lack real-time collaboration, while professional tools support collaboration but do not incorporate educational feedback mechanisms. Intelligent tutoring systems, in turn, focus on feedback but are not integrated with collaborative environments.

Consequently, a significant research gap remains between distributed collaborative editing technologies and pedagogically oriented programming environments.

In particular, no existing platform simultaneously provides:

1. CRDT-based real-time collaborative code editing
2. automated semantic analysis of learner code for adaptive hint generation
3. a child-adapted programming environment suitable for programming education
4. deployment models compatible with sovereign or on-premises infrastructure

Addressing this gap motivates the architectural approach proposed in this work.

The study is guided by the following research questions:

RQ1: How can CRDT-based real-time synchronization and automated hint generation be integrated without degrading system responsiveness?

RQ2: What architectural separation of concerns enables scalable collaboration infrastructure while supporting pedagogical analysis of code?

RQ3: To what extent does an integrated collaborative programming environment improve usability and perceived learning support compared with fragmented toolchains currently used in online programming courses?

III. MATERIALS AND METHODS

A. Proposed approach

The solution is a web-based educational platform that provides: (1) a shared code editor with real-time synchronization via CRDT (Yjs), awareness (cursors, selections), and session management; (2) a hint service that builds an AST (e.g., Tree-sitter), detects a defined set of novice errors, and returns child-adapted hints (templates and optional LLM) with visual attachment to the code; (3) authentication, project/session registry, gamification, and a single REST/WebSocket API. The design is structured in three bounded contexts: Collaborative Editing (BC-1), Pedagogical Analysis (BC-2), and Learning Management (BC-3). BC-1 and BC-2 use different runtimes (Node.js for real-time, Python for analysis) and are deployed as

separate services; BC-3 is implemented in the API Gateway and LMS module. Communication across contexts goes through the API Gateway or events; domain models are not shared between BCs.

B. Architectural challenges addressed

The design addresses three architectural challenges that arise when combining real-time collaboration and pedagogical hint generation in one platform. First, both sync latency and hint response time must stay within bounds (e.g., p95 below 100 ms and 2 s respectively) while sharing the same deployment; this is achieved by separating the Realtime Sync Service from the Hint Service and ensuring that the hint path does not block the CRDT path. Second, the hint pipeline (AST construction, error detection, hint generation) must not add latency to the propagation of CRDT updates; debouncing on the client and stateless hint requests keep the two concerns decoupled. Third, the real-time layer must scale horizontally (multiple Realtime Sync replicas, Redis Pub/Sub) without tight coupling to the hint service or the learning-management layer; the API Gateway and bounded contexts enforce this separation.

C. Models and algorithms

CRDT and synchronization. The shared document is a Yjs Y.Doc (YATA-based CRDT). Each edit produces an Update sent over WebSocket to the Realtime Sync Service, applied to the server-side Y.Doc, and broadcast to all clients in the session. For horizontal scaling, the server publishes updates to a Redis channel per session; all nodes subscribed to that channel forward updates to their connected clients. Clients buffer updates locally when disconnected and replay them on reconnect; CRDT guarantees convergence. Awareness is carried in a separate channel. Hint pipeline. The client (or gateway) sends the current code to the Hint Service. The service parses with Tree-sitter, builds an AST, and runs an Error Detector to classify errors (e.g., syntax/unmatched_paren, semantic/undefined_variable). The Hint Generator produces a short, child-appropriate message and optional visual cue, using a template bank and, when configured, an LLM; the response is constrained to avoid technical jargon. The Hint Personalizer can take into account the learner's error history. Libraries: Yjs (CRDT), Tree-sitter (parsing); choice is justified by maturity, editor integration, and suitability for real-time and analysis workloads.

D. Pedagogical Design and Psychological Considerations

The platform design incorporates pedagogical principles that are relevant for teaching programming to novice learners. Research in computer science education indicates that beginners benefit from concise feedback, contextual explanations, and interfaces that minimize unnecessary cognitive load.

In many development environments, compiler or interpreter messages are difficult for beginners to understand because they rely on technical terminology and assume prior knowledge of programming concepts. To address this limitation, the proposed platform generates short hints formulated in simplified language and linked directly to the location of the detected error. The intention is to help learners correct mistakes without interrupting their workflow or switching to external documentation.

Another design objective is to reduce context switching during collaborative programming sessions. Instead of requiring students to leave the coding environment to seek assistance, the platform provides contextual feedback directly within the editor interface. This approach supports continuous engagement with the task and allows teachers to focus on higher-level guidance rather than routine error explanations.

Motivational aspects are also considered. Programming errors are presented as part of the learning process rather than as failures. Positive reinforcement and optional gamification elements are intended to encourage experimentation and reduce the anxiety often experienced by beginners when encountering errors.

These pedagogical design decisions are reflected in the architecture of the Hint Service and will be evaluated during future usability studies involving both learners and instructors.

E. System design

The system is decomposed into three bounded contexts (Fig. 1). BC-1 (Collaborative Editing) comprises the Web Code Editor (CodeMirror 6 with Yjs), WebSocket Handler, CRDT Engine (Yjs), Session Manager, Awareness Protocol, Snapshot Service, and Message Broadcaster (Redis Pub/Sub). BC-2 (Pedagogical Analysis) contains the AST Analyzer (Tree-sitter), Error Detector, Hint Generator, and Hint Personalizer. BC-3 (Learning Management) includes the Auth Service (JWT), User Profile and Gamification, Project Registry, Analytics Collector, and the REST API Gateway.

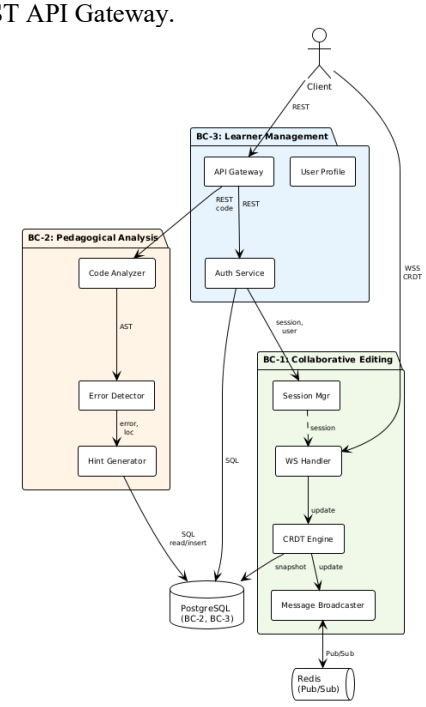

Fig. 1 Bounded contexts of the platform

The Gateway is the single entry point: it checks JWT, routes REST requests to the Hint Service and LMS logic, and proxies WebSocket connections to the Realtime Sync Service. Integration follows DDD: boundaries between BCs are respected; BC-1 uses Redis for Pub/Sub only; BC-2 and BC-3

share a PostgreSQL database; BC-1 writes snapshots but does not share domain models with BC-2 or BC-3. Fig. 2 shows the container view of the architecture: the main deployable units (Web SPA, Realtime Sync Service, Hint Service, API Gateway/LMS, PostgreSQL, Redis) and how they communicate. This level of C4 is sufficient to convey the separation of concerns and the placement of the CRDT and hint pipelines without the detail of internal components. The system design document [8] contains additional C4 diagrams (context, component), UML (e.g., sequence diagrams for the main flows), and an ER diagram of the database.

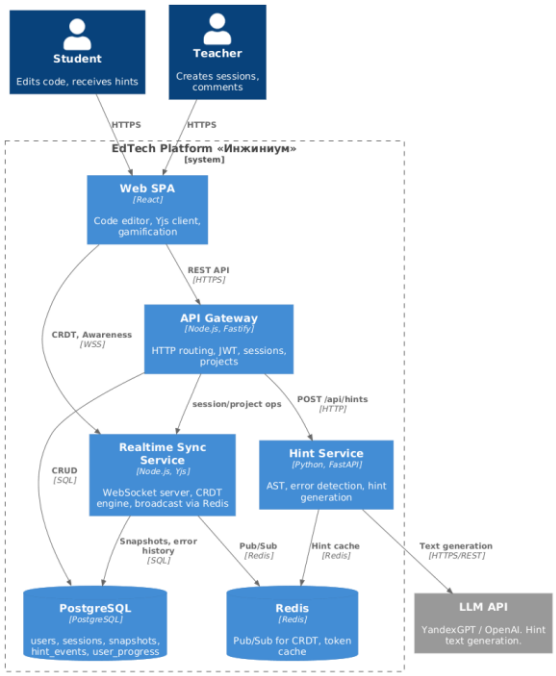

Fig. 2 Container diagram: deployable units and their relationships.

F. System architecture

The chosen style is a modular monolith with isolated services: one deployable unit for Realtime Sync (Node.js), one for Hint Service (Python), and one for API Gateway/LMS (Node.js), with shared PostgreSQL and Redis. This allows horizontal scaling of the real-time layer (multiple Realtime Sync replicas, Redis Pub/Sub for cross-node broadcast), keeps operational complexity manageable for a prototype, and allows independent evolution of the collaboration and pedagogy domains. The front end is a React SPA (CodeMirror 6, Yjs, TailwindCSS) connecting to the Gateway (HTTPS) and to the Realtime Sync Service (WSS). In production, TLS is terminated at the Ingress; all client-facing endpoints use HTTPS/WSS.

IV. Implementation, Tools and Technologies

A. Technology stack

Frontend: React 18, CodeMirror 6 (syntax highlighting, Yjs binding), Yjs 13.x, TailwindCSS. Realtime Sync Service: Node.js 20 LTS, y-websocket, uWebSockets.js. API Gateway / LMS: Node.js, Fastify 4, JWT (jsonwebtoken). Hint Service: Python 3.12, FastAPI, Tree-sitter (Python grammar), optional YandexGPT/OpenAI client. Data: PostgreSQL 16 (users, sessions, snapshots, analytics), Redis 7 (Pub/Sub, optional cache). Infrastructure: Docker, Kubernetes (e.g., Yandex Cloud), Nginx Ingress; CI/CD (e.g., GitHub Actions), SonarQube for static analysis. Choices are justified by the need for a single language on the client and real-time server (JavaScript/TypeScript), a proven CRDT library with editor integration (Yjs), an incremental parser (Tree-sitter), and the ability to scale the WebSocket layer independently.

B. Key implementation aspects

The Realtime Sync Service maintains one Y.Doc per session in memory and subscribes to the session's Redis channel. On receiving a client Update, it applies it to the local Y.Doc and publishes the same Update to Redis. The Snapshot Service periodically serializes the Y.Doc and stores it in PostgreSQL; the rollback API loads a snapshot and reinitializes the session document. The Hint Service is stateless: each request contains the code and optional user/session id; the response includes hint, location, and visualCue. Debouncing on the client (e.g., 500–1000 ms after the last keystroke) avoids excessive requests.

Simplified CRDT update handling (pseudocode)

```
   upon receive Update u from client c for session
s:
       Y.Doc[s] ← apply(Y.Doc[s], u)
       buffer ← serialize(u)
       for each node n in cluster:
           if n ≠ self then publish(buffer,
channel(s))  // e.g. Redis Pub/Sub
   upon receive buffer from channel(s):
       u ← deserialize(buffer)
       Y.Doc[s] ← apply(Y.Doc[s], u)
       broadcast u to all clients connected to s at
this node
```

Logic for applying a CRDT update and broadcasting to other nodes; convergence is guaranteed by the underlying CRDT semantics [1], [3].

C. Development and tooling

Development is done in a Docker Compose environment (all services, PostgreSQL, Redis). Testing uses Vitest (Node.js), Pytest (Python), Supertest and WebSocket clients for integration, Testcontainers for PostgreSQL/Redis, Playwright for E2E, and k6 for load tests. API contracts are described with OpenAPI.

V. Experiments, Testing and Analysis

A. Experimental Setup and Testing Strategy

The test scope will cover all three bounded contexts: Realtime Sync (CRDT, WebSocket, Session Manager, Snapshot), Hint Service (AST, Error Detector, Hint Generator), and API Gateway/LMS (auth, sessions, projects). External LLM APIs will be mocked in automated tests. The strategy will include: Unit tests — CRDT convergence, AST and error detection, hint generation, auth. Integration tests — WebSocket + Redis, Gateway → Hint Service, snapshot and rollback. System/E2E tests — Full user flows (create session, edit code, receive hint, correct error, gamification; teacher: create session,

invite, comment). Load tests — Sync latency with 10 users, hint API under concurrent requests. Usability tests — SUS with children 10–14 years; expert evaluation of hint clarity by a pedagogue. Environments: dev (Docker Compose), test (Kubernetes namespace with test DB), and a production-like stage for load and E2E. Data for testing: synthetic code samples and session scenarios; for usability, a pilot group of learners from the partner organization.

B. Evaluation Metrics

Functional: Verification of all functional requirements from the specification by checklist and acceptance tests. Performance: p95 sync latency < 100 ms with 10 concurrent users in one session; p95 hint response time < 2 s. Correctness: CRDT convergence in unit tests (concurrent inserts/deletes); no data loss after simulated WebSocket disconnect and reconnect (within a defined time window, e.g., 5 minutes). Security: All endpoints over HTTPS/WSS; HSTS; JWT validation. Usability: Mean SUS $\geqslant$ 70 in the target age group; expert rating of hint clarity (e.g., $\geqslant$ 80% of hints rated $\geqslant$ 4/5); overall user satisfaction (e.g., $\geqslant$ 4.0/5). Quality: Unit test coverage $\geqslant$ 80% for CRDT engine, AST analyzer, Error Detector, Hint Generator, Auth Service. These metrics are chosen to align with the non-functional requirements and with accepted practice in usability and ITS evaluation [7], [8].

C. Planned Evaluation and Expected Outputs

At the current stage of the project, the platform is under active development and full empirical evaluation has not yet been completed. Nevertheless, a structured evaluation framework has been defined to assess both the technical performance and the educational usability of the system once the prototype becomes stable.

The evaluation will include several complementary components. First, performance testing will measure synchronization latency and system responsiveness during collaborative editing sessions. Particular attention will be given to the scalability of the CRDT-based synchronization layer under multiple concurrent users.

Second, the hint generation pipeline will be evaluated in terms of response time and accuracy of error detection. Test datasets consisting of typical beginner programming errors will be used to assess the reliability of the AST-based analysis.

Third, usability studies will be conducted with the target user group. These studies will focus on perceived usability, clarity of hints, and overall interaction quality. The System Usability Scale (SUS) will be used to obtain a standardized usability score, and expert evaluation by instructors will assess the pedagogical usefulness of automatically generated hints.

The results of these evaluations will be presented in the form of performance graphs, usability metrics, and qualitative feedback from pilot classroom sessions.

D. Analysis and discussion

Although the empirical evaluation is still in progress, the proposed architecture suggests several potential advantages compared with existing solutions.

First, the integration of CRDT-based synchronization with a pedagogical hint generation pipeline creates the possibility of supporting real-time collaborative learning without sacrificing automated feedback. In existing practice, collaborative editing and pedagogical assistance are typically provided by separate tools. Bringing these capabilities into a single environment may reduce technological fragmentation during online programming lessons.

Second, the architectural separation between the collaboration layer and the pedagogical analysis service allows each component to evolve independently. This separation is particularly important because real-time synchronization requires extremely low latency, while semantic analysis of code may involve computationally heavier operations.

Third, the use of domain-driven decomposition makes the architecture easier to extend. Additional pedagogical services—such as learning analytics or adaptive task recommendation—can be integrated without modifying the core collaboration infrastructure.

The evaluation planned for future work will determine whether these architectural decisions translate into measurable improvements in system performance and usability.

VI. Discussion

A. Interpretation in broader context

The design is intended to show that CRDT-based real-time collaboration and AST-driven hint generation can be integrated in one platform while meeting latency and usability targets. The pattern is relevant for EdTech providers and institutions that need a single, sovereign or on-premises environment for collaborative programming education. The separation of Realtime Sync, Hint Service, and Gateway allows incremental adoption and future evolution of the hint backend (e.g., richer LLM integration) without changing the collaboration core.

B. Limitations

The current work focuses on architectural design and a prototype implementation. Several limitations should therefore be noted.

First, the hint generation module currently targets a limited set of programming languages and common beginner error patterns. Supporting additional languages will require extending the grammar definitions and analysis rules.

Second, the initial evaluation will involve a pilot group from a single educational organization. While this setting is sufficient for early validation of the concept, broader studies across different educational contexts will be required.

Finally, the present study evaluates the architecture and prototype rather than long-term learning outcomes. Assessing the impact of the platform on programming skill development will require longitudinal classroom studies.

C. Threats to validity

Internal: Network conditions or deployment (e.g., different cloud region) can affect latency; tests will be run in a controlled environment and documented. External: The partner organization and age group may not represent all educational contexts. Construct: SUS and expert ratings are standard but

subjective; additional objective metrics (e.g., task completion time, error correction rate) could be added in future work.

D. Practical applicability

The system is designed to be deployed as a single-tenant or multi-tenant web application behind an organization's identity provider. The API Gateway and OpenAPI specification allow integration with existing LMS or assignment systems. The architecture supports deployment in sovereign or air-gapped environments without dependence on external SaaS for core collaboration and hint logic.

VII. Conclusion and Future Work

This paper presented the design of a web-based educational platform that integrates real-time collaborative programming with automated pedagogical feedback. The proposed system combines CRDT-based synchronization, semantic code analysis, and learning-management functionality within a unified architecture.

The main contribution of the work lies in the architectural integration of two research directions that are usually treated separately: distributed collaborative editing and intelligent tutoring for programming. By introducing a domain-driven decomposition into three bounded contexts—Collaborative Editing, Pedagogical Analysis, and Learning Management—the platform demonstrates how real-time collaboration infrastructure can be combined with automated feedback mechanisms while maintaining strict performance constraints.

The proposed architecture also introduces an evaluation framework that combines distributed systems metrics with usability and pedagogical indicators. This framework enables a more comprehensive assessment of collaborative learning systems, addressing both technical reliability and learner experience.

To the best of our knowledge, no previous work has documented architecture that combines CRDT-based collaborative programming with automated pedagogical hint generation in a single deployable system.

Future work will focus on expanding the hint generation capabilities to additional programming languages, conducting large-scale usability studies with learners and instructors, and investigating the impact of integrated collaborative environments on programming learning outcomes. Further research may also explore adaptive feedback strategies based on learner behavior and the integration of more advanced code-analysis techniques.

Acknowledgements

Thanks are due to the educational organization "Inzhinium" for domain expertise and for agreeing to participate in pilot evaluation. Gratitude is expressed to Alexander Breyman (HSE) for his guidance and feedback on the project.

References

[1] Sun, C., Sun, D., Agustina, & Cai, W. (2018). Real differences between OT and CRDT for co-editors. arXiv preprint arXiv:1810.02137).

[2] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "Conflict-free replicated data types," in Proc. 13th Int. Symp. Stabilization, Safety, Security Distrib. Syst. (SSS), LNCS, vol. 6976. Berlin, Germany: Springer, 2011, pp. 386–400.

[3] "Yjs: CRDTs for collaborative editing." [Online]. Available: https://docs.yjs.dev/

[4] C. Sun, D. Sun, Agustina, and W. Cai, "Real differences between OT and CRDT for co-editors," arXiv:1810.02137 [cs.DC], Oct. 2018.

[5] T. Crow, A. Luxton-Reilly, and B. Wuensche, "Intelligent tutoring systems for programming education," in Proc. 20th Australasian Comput. Educ. Conf. (ACE), 2018, pp. 81–90.

[6] T. Delev and D. Gjorgjevikj, "Static analysis of source code written by novice programmers," in Proc. IEEE Global Eng. Educ. Conf. (EDUCON), 2017, pp. 825–830.

[7] B. Ericson and M. Guzdial, "Helping novice programmers succeed with subgoal labeling," in Proc. 52nd ACM Tech. Symp. Comput. Sci. Educ. (SIGCSE), 2021, pp. 343–349.

[8] J. Brooke, "SUS: a 'quick and dirty' usability scale," in Usability Eval. Ind., P. W. Jordan et al., Eds. London, U.K.: Taylor & Francis, 1996, pp. 189–194.

[9] V. Pugach, "System design and test plan: educational platform for collaborative work with CRDT and AI hint generation," HSE, Moscow, Tech. Rep., 2026.

[10] M. Resnick et al., "Scratch: programming for all," Commun. ACM, vol. 52, no. 11, pp. 60–67, 2009.

[11] D. Weintrop and U. Wilensky, "To block or not to block, that is the question: students' perceptions of blocks-based programming," in Proc. 14th Int. Conf. Learn. Sci., 2015, pp. 127–135.

[12] V. Vernon, Domain-Driven Design Distilled. Boston, MA, USA: Addison-Wesley, 2016.

[13] E. Evans, Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston, MA, USA: Addison-Wesley, 2003.

[14] G. Stahl, T. Koschmann, and D. Suthers, "Computer-supported collaborative learning: An historical perspective," in Cambridge Handbook of the Learning Sciences, R. K. Sawyer, Ed. Cambridge, U.K.: Cambridge Univ. Press, 2006, pp. 409–426.

[15] L. Williams and R. Kessler, Pair Programming Illuminated. Boston, MA, USA: Addison-Wesley, 2003.

[16] J. W. Thomas, "A review of research on project-based learning," San Rafael, CA, USA: Autodesk Foundation, 2000. [Online]. Available: http://www.bie.org/research/study/review_of_project_based_learning_2000

[17] K. Kleppmann, "CRDTs: consistency without concurrency control," IEEE Softw., vol. 39, no. 2, pp. 18–24, 2022.